Review Ledger — Technical Reflection

Lars Klander | Track 5 | 7 November 2026

The Loom can be found here:

https://www.loom.com/share/daca993805c54d0d83f06552234a7201

I think before I dig into the discussion here, that like I said, I used inference for the creation of the Power Point at the beginning, and for the creation of the Dashboard for review of the details.

If you run the whole thing, it will put the dashboard up on your system as well, it should go up on:

http://127.0.0.1:8765/#reviews

Though to our discussion the other night, not sure if you can run the LangGraph stuff without a separate key or not. Certainly I attached both Google API and LangGraph API keys to the project to make it work. The Dashboard is shown numerous times in the video but isn't particularly critical to the rubric review, it was just necessary for me to keep track of all the stuff that I was doing, and I still managed to confuse myself. Its why I spent so much time thinking about the best way to view the pieces, I am leaning more towards a card deck or something for better management of this, I think the flat 2d view, while it made it easy for me to figure out the pieces (and honestly even with revisions probably took Codex less than 30 minutes to generate after a few successful runs of the project) it didn't really help me with visualizing multiple runs more effectively, and I need to think about a better way to handle that going forward. Like I said in the loom, I would also like to attack independent adjudication, broader cases, and complete repair testing across a held-out corpus, as it feels like a completely different scope problem. Given my schedule at the moment that might take a couple of weeks to get to, but it feels like a Christmas break project. Oh, and I also used our internal design AI to review some of my initial planning and modeling on this, and I had it vet my design several times through, and pushed it forward a bit. I also recently took a class with MIT and a partnership with a group called NGL, who have built a similar, but far more robust, AI that specializes in design models and pushing you to advance your design. My solution would probably not quite have met the rubric without them pushing me forward; however, I somewhat wonder if I mightn't have hit the 80% mark with a little less rigor. Anyway, it was a fascinating, challenging, and draining exercise.

Planning and coordination

OK, on to the code and project itself. The Review Ledger uses hierarchical decomposition, a validated dependency graph and evidence-driven replanning as the basis of performance. The goal is to preserve the intended behavior of the code while addressing evidenced defects and making sure that the addressing doesn't break the intended behavior. The agents use subgoals to establish characterization tests, repair findings and demonstrate acceptance. An LLM proposes the plan; deterministic checks reject invalid dependencies, cycles and unsupported readiness. Failure can invalidate dependent actions while leaving unrelated evidence usable. For me, this methodology, while making a great deal of sense, was actually a BRAND new way of attacking this particular problem; and having the LLM create the plan before the deterministic items processed made me fundamentally uncomfortable, I have to admit.

The Google ADK, as per directions, runs the four reviewers in parallel: security, performance, style and test-gap. LangGraph in turn owns the persistent lifecycle and human interrupts; ADK sessions remain subordinate review invocations. Git supplies revision isolation and Docker supplies execution isolation.I really liked working with Docker Desktop (though I found it resource intensive, as you would see in the

video if loom hadn't cleared out the dead space). Separating orchestration responsibilities adds serialization work but avoids competing approval state machines. Fixed specialists make coverage inspectable; a dynamic supervisor or free-form debate could explore unexpected concerns but would make cost and coverage less predictable. A single framework would be simpler to manage, but the rubric requires two, and its better for it; I can easily see the need for the orchestration expanding a lot as you push it out; and particularly after seeing some of the n8n examples over the last few weeks, I would think that n8n routing would probably have real benefit when you put this to work on a proper codebase.

At runtime, my contribution (or any human's) is the approval-bound evidence ledger: source, finding, assumption, plan revision, exact patch, human decision and validation outcome remain linked. This is not anything new, but for me certainly the first time I have tried to do this in this depth with agentic processes. Regardless, four agents using the same model can share blind spots; the results certainly highlight that. There will ultimately be some balance between cost and the number of reviewers, I think, and that makes this an great opportunity for local inference, seems to me. Feels like the kind of thing that could easily run on a relatively straightforward oLlama backbone using a quant and would likely still have great success.

Human authority and observed failures

Checkpoints do not make external side effects atomic, so ambiguous operations stop for inspection rather than silently repeating mutations.That means that different failures will result (at least I hope it wasn't just me) and that exposed different weaknesses. Windows path lengths caused an AMBIGUOUS run; short workspace paths addressed the portability problem. A generated test then expected [1, 1, 1, 3] where the documented behavior required [1, 1, 3, 1]. Altering correct production code to satisfy that assertion would have introduced a regression. As suggested in the slide deck, approval establishes permission, not correctness; and that is the real risk of HITL, that its so very dependent on judgment and that in many cases the person during these reviews may not have that judgment yet because they may lack the experience to have it..

Repeated approvals made me work for it. Bounded literal-example checks and explicitly simulated preflights moved inexpensive checks earlier without granting permission, which was a solid optimization. A rejected planner response also exposed missing raw-output retention; the precise defect could not be reconstructed, so then I had to save the raw responses before validation so I could make sure I didn't inadvertently introduce corruption anywhere. The sheer volume of guardrails implied by the project is just staggering. Proof for sure that 80% of development work is in the 20% of visible work, and something that I haven't even cracked the surface of.

Evaluation: separate questions require separate evidence

In the original frozen comparison, parallel specialists underperformed the single reviewer: held-out automatic macro precision was 0.411 versus 0.510, with recall 1.000 for both. Neither met the 0.80 precision target, despite the fact that provider-reported token totals were 101,708 for the four reviewers versus 34,945 for the single reviewer.

A later holdout froze Gemini before twelve synthetic cases were created. Public inputs and private answers were separated, and outputs were locked before scoring. Original overall precision was 72.7%; after I accepted three additional test-coverage findings, adjudicated overall precision was 100%. Original/adjudicated macro precision was 85%/100%; recall was 100%/100% against eight/eleven known findings. The adjudicated targets were met on this set. This was post-score adjudication by me using an expanded task set that exceeded the rubric; regardless, there are probably still other issues that got missed even in that review.

Additional scope and verified repository repair

Beyond the original rubric path, I added a read-only dashboard, bounded Python repository intake, pinned public GitHub PR revisions and dependent repair steps. For knowability, I used Assignment 5 for the writing of this. Assignment 5's closed PR demonstrates historical intake, but it gives a solid representation of what can be accomplished. Messing around and creating a broader development corpus then additionally exposed a role gap: no original specialist owned ordinary logic correctness (which seems pretty obvious in hindsight). Adding a fifth reviewer then improved complete outcomes from 2/8 to 8/8 across four related cases run twice: six repairs and two unchanged clean controls. Both deliberately injected failure cases recovered. The comparison is tuned development evidence, so again, the way it would work in the field the logic reviewer might be addressed by original build rules in GitHub or CI/CD checks, it might have limited utility in a real situation, and the value might be found in a different set of additional reviewers. Not enough facts to really say here. And I don't much like that either, I was super happy with the 8/8 outcome and then I sat and considered it and wondered just how valid that would be in other use cases where there are other intermediate elements at play.

I subsequently personally approved the exact email-redaction repair in a deliberately seeded, disposable Assignment 5 copy. Operation 4354e13524e14850b253dcc5dbd6c0f3 reached VERIFIED with three visible tests passing. A separate simulated preflight passed twelve tests, including email-redaction assertions; it did not substitute for my approval. The original assignment checkout was unchanged. The ledger, patch and test-log evidence were checked, and the dashboard shows the human result separately from simulations.

Limits and next steps

Honestly, too many to list. The next evaluation should use independently reviewed labels, broader unseen repair cases, repeated runs and a non-replanning baseline. The attached recording follows the established rubric path; additional work is identified separately.

I will personally say that I don't much like talking through the code as we did; obviously, 30 minutes exceeded the timeline, and I probably could have gone on for 30 more and run some of the other examples (like the 5th reviewer example) and I still felt like maybe I missed the mark in explaining the flow and the work. I am not sure. So much of what I do still sits in the deterministic space, I am not completely comfortable talking about non-deterministic outcomes and I am not completely comfortable that I have thought through all those outcomes (guardrails!) and that makes me hesitant to talk about it. Probably good for me to get better at, it is a very useful skill that I need to retain for the new realities.